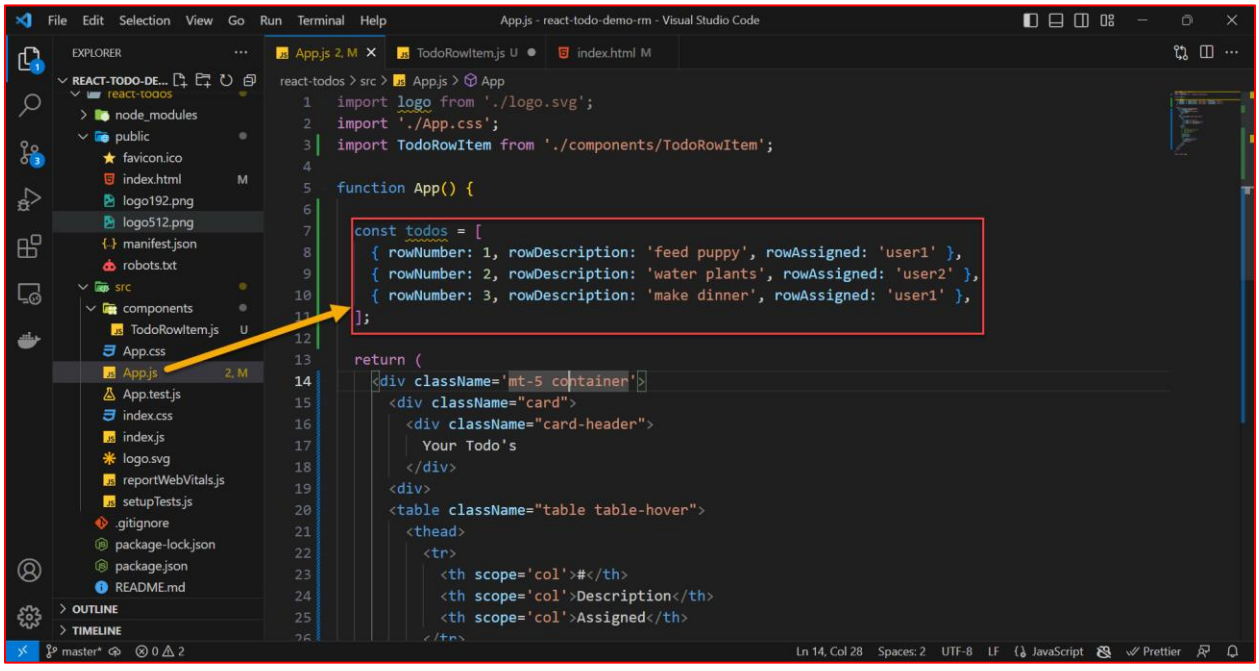
Passing Data To React Components Dynamically

In the last lesson, we built the `TodoRowItem` component that houses two identical components, holding the row number, row description, and row assigned - all static and hardcoded within the component. As we move forward, our goal is to make these elements dynamic and to reflect unique data in each row of our to-do list. Let's dive into the details.

Creating Dynamic Data

To kick things off, we need to define an array of to-do items that we want to display in our table. In `App.js`, just above the `return` statement and right under the function declaration, let's create this array:

```
const todos = [
  { rowNumber: 1, rowDescription: 'feed puppy', rowAssigned: 'user1' },
  { rowNumber: 2, rowDescription: 'water plants', rowAssigned: 'user2' },
  { rowNumber: 3, rowDescription: 'make dinner', rowAssigned: 'user1' },
];
```

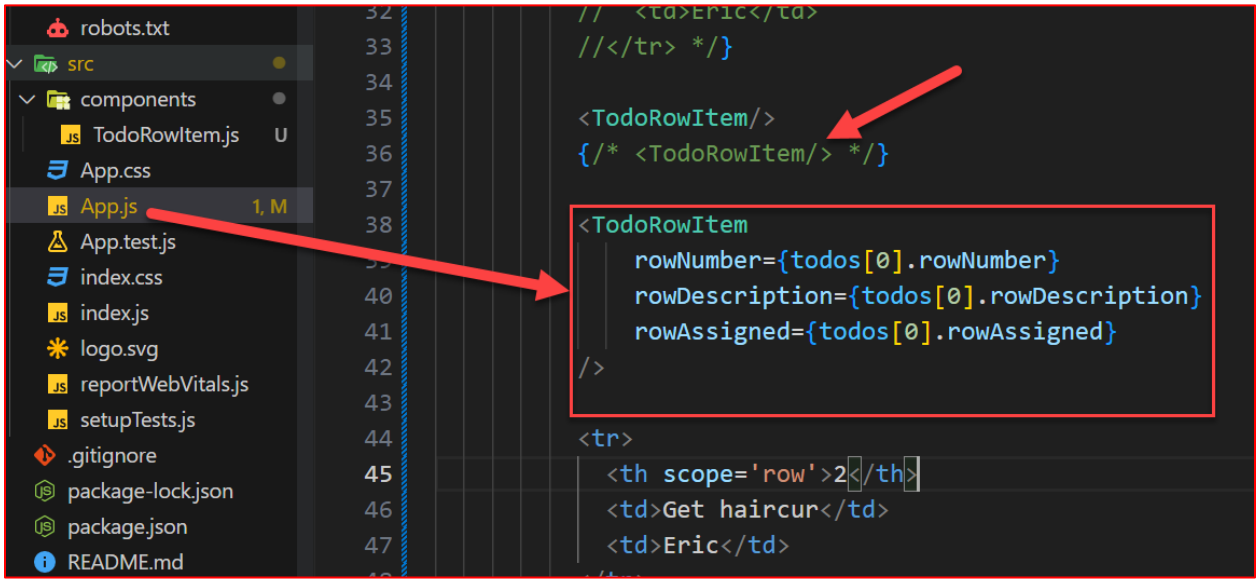


Now we have an array `todos`, holding three JavaScript objects representing the tasks we want to list.

Passing Data to Components

Next, let's navigate down to our `TodoRowItem`. We'll keep just one for now, and we'll pass one of these objects to it. Here's how you can do that:

```
<TodoRowItem
  rowNumber={todos[0].rowNumber}
  rowDescription={todos[0].rowDescription}
  rowAssigned={todos[0].rowAssigned}
/>
```



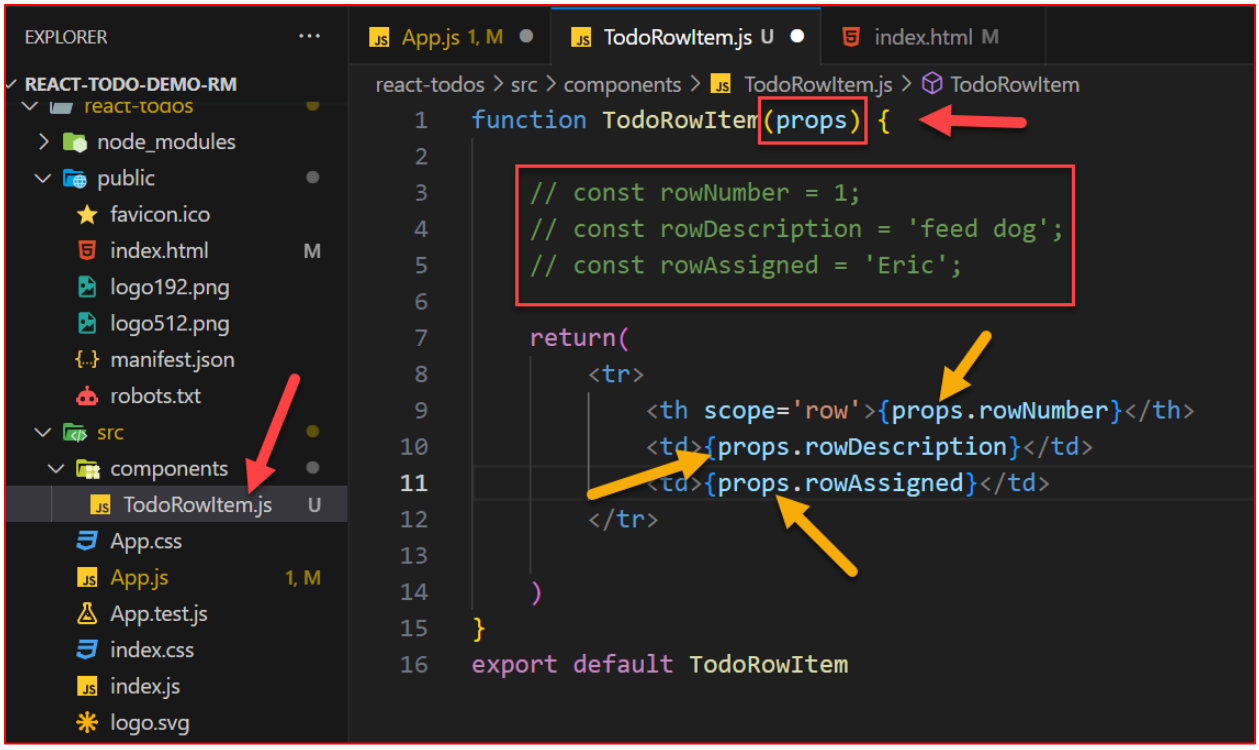
Notice the structure: we have our `TodoRowItem` with three properties inside: `rowNumber`, `rowDescription`, and `rowAssigned`. But this alone won't suffice, as we have to modify the component itself to utilize these properties.

Utilizing Props in Components

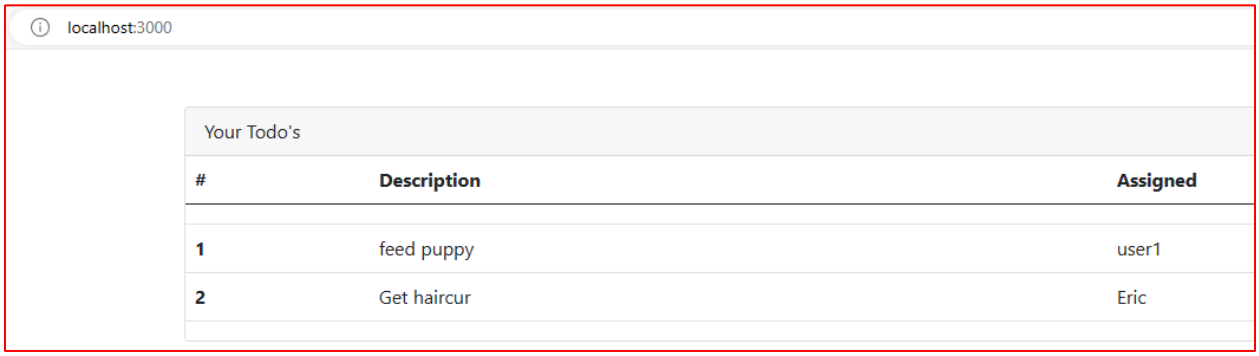
React has a unique feature allowing you to pass only one element when passing data into a function component. Typically, you'll use something called `props`. "Props" stands for properties, and it contains key-value pairs for every item passed in.

Here's how you can adjust the `TodoRowItem` component to use these props:

```
function TodoRowItem(props) {
  return (
    <tr>
      <td>{props.rowNumber}</td>
      <td>{props.rowDescription}</td>
      <td>{props.rowAssigned}</td>
    </tr>
  );
}
```

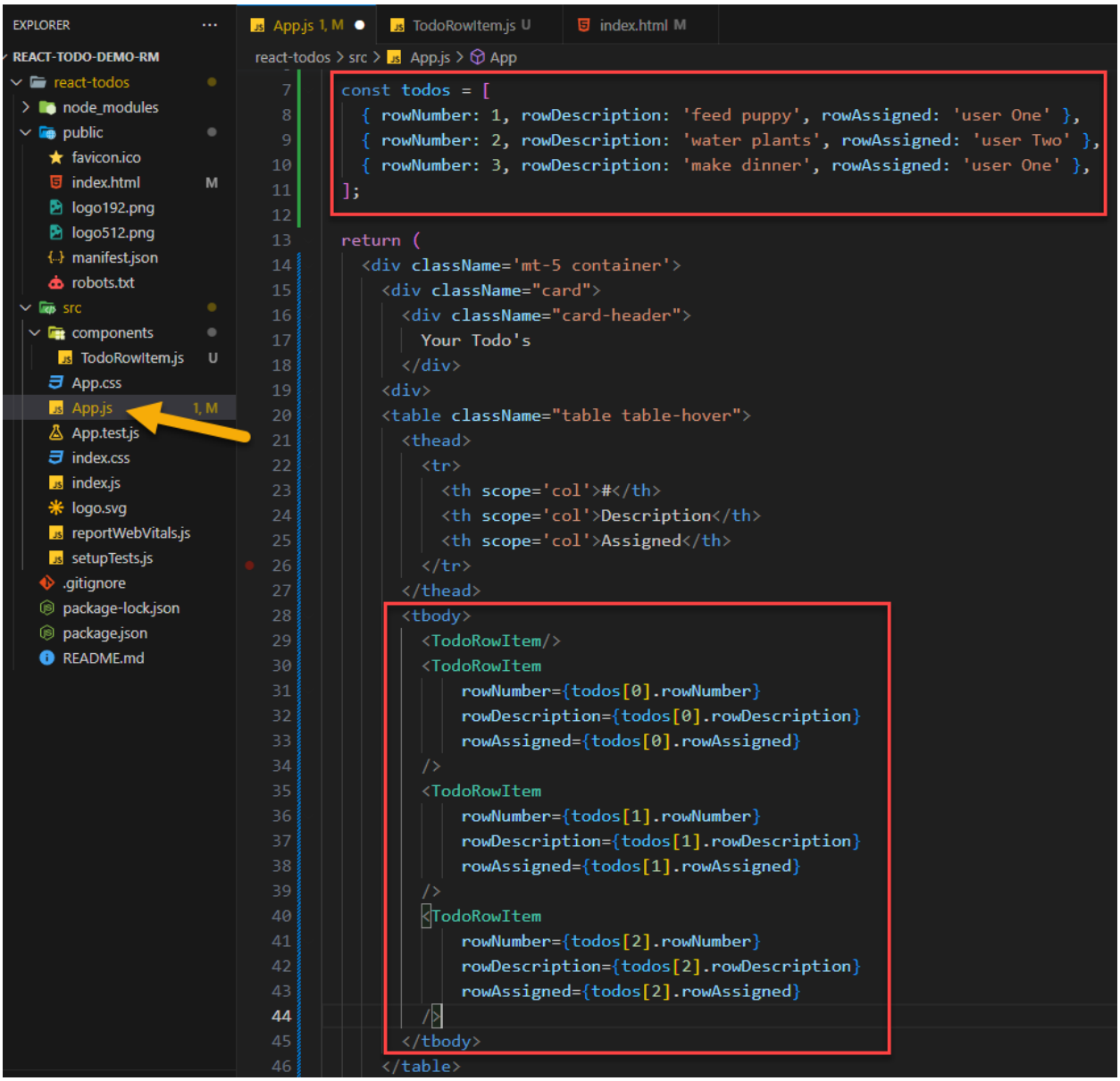


Save and refresh your application, and you'll see "Feed Puppy" displayed, sourced directly from our `todos` array.

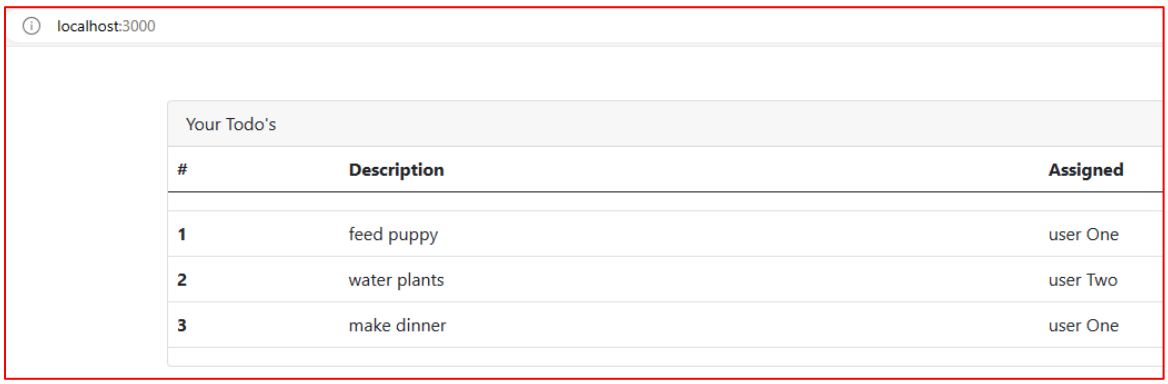


Implementing Reusable Components

Remember, the beauty of React lies in component reusability. Let's delete the hard-coded table row in `App.js` and call the `TodoRowItem` component two more times, updating the indices to pass the other two objects from our `todos` array.



With these changes, refreshing your browser will now reveal three unique items: feeding the puppy, watering the plants, and making dinner, each assigned to different users.



Key Notes on Components and Props

One vital aspect to grasp is that components cannot access data from other components without explicitly receiving them through props. It underscores the importance of the parent component passing the required data to the child components.

Summary

We've successfully explored how to utilize props in React, enabling us to pass dynamic data from our main `App.js` file to the child component `TodoRowItem`. Through this exercise, our understanding of React's reusability and dynamic rendering has deepened.